

PYTHON PROJECT ANALYZER

Code Architecture Detailed Report

An in-depth breakdown of the five core modules:

Scanner · Extractor · Analyzer · Builder · Main

Generated · May 2026

Introduction

What is the Python Project Analyzer?

The Python Project Analyzer is a modular, multi-file tool that automatically scans a Python codebase, extracts structural metadata (functions, classes, imports), computes complexity metrics, and produces a polished Markdown README — all from the command line.

1

scanner.py

Walks directory tree,
finds .py files

2

extractor.py

Parses AST — pulls
functions, classes,
imports

3

analyzer.py

Enriches & aggregates
data into master dict

4

builder.py

Renders Markdown
README from master dict

5

main.py

CLI entry point — wires
everything together

Module 1 · Core Data Extraction — analyzer.py



Objective

Build a comprehensive master dictionary for the entire project by orchestrating per-file analysis, accumulating totals, and producing a project-level summary.



Input Format

project_name (str) · file_paths (list of .py path strings) · extract_func (callable that returns raw dict with functions / classes / imports keys).



How It Works

Iterates file_paths → calls analyze_file() per path → enriches functions & classes → computes per-file metrics (avg/max complexity, counts) → appends to master dict → calls build_summary() to aggregate project-wide stats.



Output Format

Returns master_dict: { project_name, created_at, total_files/functions/classes/imports, files: [{file_name, path, extension, size_bytes, checksum, classes, functions, imports, metrics}], summary: {...} }

Module 2 · Enrichment & Sorting — analyzer.py helpers

Objective

Transform raw AST data into richly annotated entries by stamping derived flags onto every function and class, and ordering them for easy human navigation.

Input Format

Raw list of function dicts ({name, args, line_count, docstring, complexity}) and class dicts ({name, methods, docstring}) from the extractor.

How It Works

sort_functions() sorts public-first then by descending complexity. enrich_function() adds is_private, is_dunder, complexity_label (simple/moderate/complex/very_complex), arg_count. enrich_classes() adds method_count and has_init flag. dedupe_imports() preserves insertion order while removing duplicates.

Output Format

Sorted, annotated list of functions with extra keys (is_private, is_dunder, complexity_label, arg_count) and classes with (method_count, has_init).

Module 3 · File Utilities — analyzer.py helpers



Objective

Safely retrieve low-level file metadata and handle parsing failures without crashing the pipeline — every file gets an entry even if it cannot be read.



Input Format

A filesystem path string (path). Optionally an algorithm name ('md5', 'sha256') for compute_checksum(), and an Exception object for build_error_entry().



How It Works

safe_file_size() wraps os.path.getsize() — returns -1 on OSError. compute_checksum() reads the file in 8 KB chunks, hashing incrementally. build_error_entry() creates a minimal stub dict so the rest of the pipeline can continue without a KeyError.



Output Format

size_bytes (int, -1 on error) · checksum (hex str or None) · error_entry dict: {file_name, path, error, classes: [], functions: [], imports: []}.

Module 4 · Search & Query — analyzer.py

Objective

Provide fast, reusable query functions over the completed master dict so callers can slice and inspect results without re-parsing the codebase.

Input Format

master_dict (the fully built project dict) plus a query parameter: function name (str), import name (str), complexity threshold (int), or no extra param for group_by.

How It Works

find_function() scans all files for name matches. find_files_by_import() filters by imports list.
get_complex_functions() filters by complexity $\geq$ threshold then sorts descending.
group_files_by_extension() buckets files using defaultdict keyed on extension.

Output Format

List of {file, function} match dicts · List of file_name strings · Sorted list of {file, function} dicts · Dict of {'py': [file_entries]} grouped by extension.

Module 5 · Export — analyzer.py + builder.py

Objective

Persist analysis results as machine-readable JSON and human-readable Markdown — enabling both downstream tooling integration and immediate documentation delivery.

Input Format

`export_to_json / export_summary_report`: `master_dict + output_path (str)`. `build_readme` (`builder.py`): `master_dict` with fully enriched files list.

How It Works

`export_to_json` uses `json.dump` with `indent=2` and `default=str` for datetime safety.
`export_summary_report` formats a plain-text report with complexity distribution, top imports & file types. `build_readme` constructs GitHub-flavored Markdown with collapsible `<details>` sections, function/complexity tables, emoji indicators (🟢 🟡), a tree view, and anchor-based navigation.

Output Format

`README_GENERATED.md` (Markdown with tree, TOC, per-file function tables) · `project_data.json` (full nested dict) · `summary_report.txt` (plain-text stats digest).

Conclusion

A Complete, Modular Pipeline

-  `scanner.py` — discovers every `.py` file, respecting common ignore rules
-  `extractor.py` — uses Python's own AST for accurate, runtime-free parsing
-  `analyzer.py` — enriches, sorts, and aggregates across the entire project
-  `builder.py` — converts structured data into polished, navigable Markdown
-  `main.py` — single CLI command wires `scan` → `extract` → `analyze` → `build`